

Blockme

No-Turn-Back Social Media Blocker — Implementation Plan

Cross-Device • VPN-Based • Tamper-Proof

Blockme - Cross-Device Social Media Blocker: Implementation Plan

Context

The user wants a self-hosted product that blocks social media on ALL their devices (phones, tablets, computers) — including native apps (Instagram app, TikTok app, etc.) — everywhere they go: home WiFi, mobile data (4G/5G), and any network. The blocker must be hard to bypass (password-protected with a cooldown delay) and support granular per-platform control including sub-feature blocking (e.g., block YouTube Shorts but allow YouTube long-form videos).

Why DNS-only (via router) isn't enough: Router DNS only works on home WiFi. When the user is on 4G/5G or a different WiFi, the phone ignores the home router's DNS entirely. Native apps also have their own network stacks and may bypass browser-level controls.

Architecture: VPN + DNS Blocking

The solution uses **WireGuard VPN** as the delivery mechanism:

```
graph TD
    User[User's Device (iPhone, Android, Laptop)] -- "WireGuard VPN tunnel (encrypted)" --> Blockme[Blockme Server (cloud VPS or home server with port forwarding)]
    Blockme --> WireGuard[WireGuard VPN interface (port 51820 UDP)]
    Blockme --> DNS[DNS blocking server (port 53, internal VPN interface only)]
    DNS --> Social[Social media queries → returns 0.0.0.0]
    DNS --> Other[All other queries → forwarded to 8.8.8.8]
    Blockme --> API[Express API server (port 3000)]
    Blockme --> Dashboard[React dashboard]
```

How devices connect: User installs the free WireGuard app on each device, scans a QR code generated by Blockme → instantly connected. The WireGuard config sets the device's DNS to the Blockme server's VPN IP, so ALL DNS queries (from browsers AND native apps) go through the blocker regardless of what network they're on.

Result: Instagram app blocked on 4G. TikTok blocked on hotel WiFi. Everything, everywhere, including native apps.

Tech Stack

- **Backend:** Node.js + Express + TypeScript
- **VPN:** WireGuard (system package) — managed via `wg / wg-quick` CLI shell calls
- **DNS:** `dns2` npm package (UDP DNS server on port 53 of WireGuard interface)
- **Database:** SQLite + `better-sqlite3`
- **Frontend:** React + TypeScript + Vite + Tailwind CSS v4
- **Auth:** `bcrypt` + `jsonwebtoken` (JWT)
- **Scheduling:** `node-cron`
- **QR codes:** `qrcode` npm package (for WireGuard device setup)

Directory Structure

```
/home/user/Blockme/
├─ package.json                # npm workspaces root
├─ .gitignore
├─ README.md
├─
├─ server/
│  ├─ package.json
│  ├─ tsconfig.json
│  └─ src/
│     ├─ index.ts              # Express app + startup
│     ├─ dns-server.ts         # DNS interception engine (on WireGuard interface)
│     ├─ wireguard.ts          # WireGuard peer management (shell calls to wg)
│     ├─ blocklist.ts          # Domain list + matching algorithm
│     ├─ scheduler.ts          # Cron-based schedule checker
│     ├─ db/
│     │  ├─ index.ts           # SQLite singleton
│     │  ├─ schema.ts          # Table creation + seed data
│     │  └─ queries.ts         # All typed SQL functions
│     └─ routes/
│        ├─ auth.ts            # /api/auth/*
│        ├─ settings.ts        # /api/settings (incl. cooldown logic)
│        ├─ devices.ts         # /api/devices/* (VPN peer management + QR)
│        ├─ blocklist.ts       # /api/blocklist/*
│        ├─ schedules.ts       # /api/schedules/*
│        └─ stats.ts           # /api/stats/*
├─
├─ client/
│  ├─ package.json
│  ├─ tsconfig.json
│  ├─ vite.config.ts
│  ├─ postcss.config.ts
│  ├─ index.html
│  └─ src/
│     ├─ main.tsx
│     ├─ App.tsx
│     ├─ api/client.ts
│     ├─ contexts/AuthContext.tsx
│     ├─ pages/
│     │  ├─ LoginPage.tsx
│     │  ├─ DashboardPage.tsx  # Status + stats overview
│     │  └─ DevicesPage.tsx    # Add device → QR code setup
```

```

├── BlocklistPage.tsx    # Per-platform + URL rule toggles
├── SchedulesPage.tsx
├── SettingsPage.tsx    # Cooldown, password, setup guide
├── components/
│   ├── Layout.tsx
│   ├── ProtectedRoute.tsx
│   ├── BlockToggle.tsx    # Master toggle + cooldown countdown
│   ├── PlatformCard.tsx   # Per-platform toggle + sub-rules
│   ├── DeviceQR.tsx       # QR code display + download config
│   └── StatsChart.tsx
├── extension/           # Optional browser extension
│   ├── manifest.json     # Chrome MV3 / Firefox compatible
│   ├── background.ts     # URL-pattern blocking (YouTube Shorts etc.)
│   ├── popup/
│   │   ├── popup.html
│   │   └── popup.ts

```

Database Schema

```

-- Admin auth
CREATE TABLE admin (id INTEGER PRIMARY KEY, password_hash TEXT NOT NULL);

-- Key/value settings
-- Keys: blocking_enabled, upstream_dns, protection_enabled,
--        cooldown_minutes (default '30'), disable_requested_at (timestamp or null),
--        server_ip, wg_server_private_key, wg_server_public_key
CREATE TABLE settings (key TEXT PRIMARY KEY, value TEXT NOT NULL);

-- Platform registry
CREATE TABLE platforms (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL,          -- 'YouTube', 'Instagram'
  category    TEXT NOT NULL,          -- 'video', 'social', 'messaging', 'professional'
  enabled     INTEGER NOT NULL DEFAULT 1
);

-- DNS-level domain rules
CREATE TABLE domains (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  platform_id INTEGER REFERENCES platforms(id), -- NULL = custom
  domain      TEXT NOT NULL UNIQUE,
  is_custom   INTEGER NOT NULL DEFAULT 0,
  enabled     INTEGER NOT NULL DEFAULT 1
);

-- URL-pattern rules for browser extension (sub-platform granularity)
CREATE TABLE url_rules (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  platform_id INTEGER REFERENCES platforms(id),
  name        TEXT NOT NULL,          -- 'YouTube Shorts'
  pattern     TEXT NOT NULL,          -- '*youtube.com/shorts*'
  enabled     INTEGER NOT NULL DEFAULT 1
);

-- WireGuard connected devices
CREATE TABLE devices (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL,          -- 'iPhone', 'Work Laptop'
  public_key  TEXT NOT NULL UNIQUE,   -- WireGuard public key (base64)

```

```

    vpn_ip      TEXT NOT NULL UNIQUE,    -- Assigned VPN IP, e.g. 10.13.13.2
    created_at   TEXT NOT NULL DEFAULT (datetime('now')),
    last_seen    TEXT
);

-- Schedule rules
CREATE TABLE schedules (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT NOT NULL,
    enabled     INTEGER NOT NULL DEFAULT 1,
    days_mask   INTEGER NOT NULL,        -- bitmask: bit0=Sun...bit6=Sat
    start_time  TEXT NOT NULL,          -- 'HH:MM'
    end_time    TEXT NOT NULL,          -- 'HH:MM'
    created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Block statistics
CREATE TABLE stats_daily (
    date        TEXT NOT NULL,
    domain      TEXT NOT NULL,
    count       INTEGER NOT NULL DEFAULT 0,
    PRIMARY KEY (date, domain)
);

CREATE TABLE stats_totals (
    domain      TEXT PRIMARY KEY,
    count       INTEGER NOT NULL DEFAULT 0,
    last_blocked TEXT NOT NULL DEFAULT (datetime('now'))
);

```

Pre-seeded platforms and domains (partial list):

Platform	Category	Domains
Twitter/X	social	twitter.com, x.com, t.co, twimg.com
Instagram	social	instagram.com, cdninstagram.com, i.instagram.com
Facebook	social	facebook.com, fb.com, fbcdn.net, messenger.com
TikTok	video	tiktok.com, tiktokcdn.com, musical.ly, snssdk.com
YouTube	video	youtube.com,youtu.be, yimg.com, googlevideo.com
Reddit	social	reddit.com, redd.it, redditstatic.com
Snapchat	messaging	snapchat.com, sc-cdn.net, snap.com
Discord	messaging	discord.com, discordapp.com, discord.gg
LinkedIn	professional	linkedin.com, licdn.com
Pinterest	social	pinterest.com, pinimg.com
Twitch	video	twitch.tv, jtvnw.net
Threads	social	threads.net
WhatsApp	messaging	whatsapp.com, whatsapp.net
Telegram	messaging	telegram.org, t.me
Bluesky	social	bsky.app, bsky.social

Platform	Category	Domains
BeReal	social	bereal.com

Pre-seeded URL pattern rules (sub-platform granularity):

Platform	Rule Name	Pattern
YouTube	YouTube Shorts	*youtube.com/shorts*
YouTube	YouTube Reels	*youtube.com/reels*
Reddit	Reddit Live	*reddit.com/live*
Twitter/X	Twitter Trending	*twitter.com/i/trending*

WireGuard Management (`server/src/wireguard.ts`)

Manages WireGuard peers via shell calls to `wg` and `wg-quick`.

```
// Generate a new device keypair (shell calls to `wg genkey | wg pubkey`)
async function generateKeypair(): Promise<{ privateKey: string; publicKey: string }>

// Add peer to WireGuard config and reload with `wg synconf`
async function addPeer(publicKey: string, vpnIp: string): Promise<void>

// Remove peer from WireGuard config
async function removePeer(publicKey: string): Promise<void>

// Generate WireGuard client config string for download/QR
function generateClientConfig(device: Device, serverPublicKey: string, serverIp: string): string
// Returns:
// [Interface]
// PrivateKey = <generated>
// Address = 10.13.13.X/32
// DNS = 10.13.13.1      ← Points to Blockme's DNS server
//
// [Peer]
// PublicKey = <server pubkey>
// Endpoint = <serverIp>:51820
// AllowedIPs = 0.0.0.0/0, ::/0    ← Route ALL traffic through VPN
// PersistentKeepalive = 25
```

Key: `DNS = 10.13.13.1` in the client config is what routes all DNS queries through the Blockme DNS server.

`AllowedIPs = 0.0.0.0/0` routes all traffic through the VPN (full tunnel mode), ensuring there is no DNS leak.

VPN IP assignment: Start at `10.13.13.2`, increment per device. Server is `10.13.13.1`.

DNS Server (`server/src/dns-server.ts`)

Listens on `10.13.13.1:53` (WireGuard interface only — not exposed publicly).

Domain matching algorithm:

```
function isDomainBlocked(queryName: string, blockSet: Set<string>): boolean {
  const name = queryName.toLowerCase().replace(/\.$/, '');
  // ...
```

```

if (blockSet.has(name)) return true;
// Walk up: 'api.instagram.com' → check 'instagram.com' → MATCH
const parts = name.split('.');
for (let i = 1; i < parts.length - 1; i++) {
  if (blockSet.has(parts.slice(i).join('.'))) return true;
}
return false;
}

```

Block response: Return A record `0.0.0.0` and AAAA record `::` with TTL=60. Both IPv4 and IPv6 blocked to prevent bypass. Short TTL so unblocking takes effect within 60 seconds.

In-memory cache: `blockSet` is a `Set<string>` cached in memory, refreshed via `refreshBlockSet()` after any domain change. Never hits DB per DNS query.

Hard Lock + Cooldown (`server/src/routes/settings.ts`)

When `protection_enabled = '1'`:

```

BLOCKING_ON
  | user clicks "Disable Blocking"
  ▼
COOLDOWN_PENDING ← sets disable_requested_at = now()
  | cooldown_minutes elapsed (default 30 min)
  ▼
CAN_DISABLE ← API allows setting blocking_enabled = '0'
  | user confirms disable
  ▼
BLOCKING_OFF

```

- GET `/api/settings` returns `{ cooldown_state, cooldown_remaining_seconds }` so dashboard can show live countdown
- Cooldown is stored as a timestamp in the DB (`disable_requested_at`), not a server-side timer — survives server restarts
- Password required to change cooldown duration or disable protection mode itself
- Rate limiting: 5 login attempts per 15 minutes (express-rate-limit)

API Endpoints

Method	Path	Auth	Description
POST	<code>/api/auth/login</code>	No	Verify password, return JWT
GET	<code>/api/auth/status</code>	No	Check token validity
GET	<code>/api/settings</code>	No	Get settings + cooldown state
PUT	<code>/api/settings</code>	Yes	Update settings / request blocking disable
PUT	<code>/api/settings/password</code>	Yes	Change admin password
GET	<code>/api/devices</code>	Yes	List VPN devices
POST	<code>/api/devices</code>	Yes	Add device (generate keypair + WG config + QR)
GET	<code>/api/devices/:id/config</code>	Yes	Download WireGuard .conf file

Method	Path	Auth	Description
GET	/api/devices/:id/qr	Yes	Get QR code as PNG
DELETE	/api/devices/:id	Yes	Remove device from VPN
GET	/api/blocklist	No	All platforms + domains + url_rules
POST	/api/blocklist/domains	Yes	Add custom domain
PATCH	/api/blocklist/domains/:id	Yes	Toggle domain enabled
DELETE	/api/blocklist/domains/:id	Yes	Remove custom domain
PATCH	/api/blocklist/platforms/:id	Yes	Toggle entire platform
PATCH	/api/blocklist/url-rules/:id	Yes	Toggle URL pattern rule
GET	/api/schedules	No	List schedules
POST	/api/schedules	Yes	Create schedule
PUT	/api/schedules/:id	Yes	Update schedule
DELETE	/api/schedules/:id	Yes	Delete schedule
GET	/api/stats/summary	No	Today/week/alltime totals
GET	/api/stats/daily?days=N	No	Per-day block counts
GET	/api/stats/domains?limit=N	No	Top blocked domains
DELETE	/api/stats	Yes	Clear all stats

Frontend Pages

Dashboard

- Large status card: "Blocking Active" / "Cooldown: 24:31 remaining" / "Blocking Disabled"
- Stats: Blocked Today / This Week / All Time
- Recharts bar chart: last 7 days of blocked requests
- Top 5 blocked domains table
- Active devices count

Devices Page ★ (new, key feature)

- List of connected devices with name, VPN IP, last seen
- "Add Device" button → form: device name → generates WireGuard config
- Shows QR code (scan with WireGuard mobile app) + download .conf button
- Step-by-step setup instructions per OS (iOS, Android, Windows, Mac)
- Remove device button (revokes access)

Blocklist

- Platform grid grouped by category (Social, Video, Messaging, Professional)
- Each platform: toggle on/off + expand to see domains + URL rules

- "Block All" / "Unblock All" buttons
- Custom domain input (add any domain)
- URL rules section: granular sub-feature toggles (YouTube Shorts, etc.)

Schedules

- Schedule cards showing active days + time range
- "New Schedule" modal: day checkboxes, time pickers, name

Settings

- Protection mode toggle + cooldown duration selector (5/15/30/60/120 min)
- Change password
- Server IP configuration (used in device configs)
- Clear statistics

Browser Extension (`extension/`) — Desktop bonus

Provides sub-platform URL-pattern blocking that DNS can't do (same domain, different paths). Not needed for mobile (VPN handles everything at the network layer).

- Chrome MV3 + Firefox compatible
- `background.ts` : Fetches enabled URL rules from API, uses `declarativeNetRequest` to block specific URL patterns
- Shows blocked count in popup
- Configurable Blockme server URL (default `http://localhost:3000`)

Bypass Hardening

1. **Cooldown timer** (default 30 min): Can't impulsively disable blocking
2. **WireGuard full tunnel** (`AllowedIPs = 0.0.0.0/0`): All traffic routes through VPN — no split tunnel leaks
3. **DNS on VPN interface only**: DNS server isn't publicly accessible — only VPN clients use it
4. **Both A + AAAA blocked**: No IPv6 bypass
5. **Subdomain wildcard matching**: All subdomains of blocked domains auto-blocked
6. **TTL=60**: Unblocking takes effect within 60s (no stale cached blocks lingering)
7. **Rate-limited login**: Prevents password brute-force

Implementation Sequence

1. Root `package.json` (workspaces: server, client, extension)
2. `server/package.json` + `server/tsconfig.json`
3. `server/src/db/schema.ts` — tables + seed (platforms, domains, URL rules, default settings)
4. `server/src/db/index.ts` — SQLite singleton + WAL mode
5. `server/src/db/queries.ts` — all typed query functions
6. `server/src/blocklist.ts` — domain list + `isDomainBlocked()` + block set cache
7. `server/src/scheduler.ts` — `isCurrentlyBlocking()` using schedules + settings
8. `server/src/dns-server.ts` — DNS engine on WireGuard interface
9. `server/src/wireguard.ts` — keypair generation, peer add/remove, config generation

10. `server/src/routes/auth.ts` — login + JWT + requireAuth middleware + rate limit
11. `server/src/routes/settings.ts` — settings CRUD + cooldown state machine
12. `server/src/routes/devices.ts` — device CRUD + WireGuard peer management + QR
13. `server/src/routes/blocklist.ts` — platform/domain/URL rule toggles
14. `server/src/routes/schedules.ts` — schedule CRUD
15. `server/src/routes/stats.ts` — stats queries + clear
16. `server/src/index.ts` — wire Express + DNS + WireGuard together
17. `client/` setup: `package.json`, Vite, Tailwind v4 configs
18. `client/src/` — AuthContext, `api/client.ts`, `App.tsx` routing
19. `client/src/pages/` — Dashboard, Devices, Blocklist, Schedules, Settings, Login
20. `extension/` — manifest, background service worker, popup
21. `README.md` — server setup instructions, WireGuard install, systemd service

Server Requirements

- Linux machine (cloud VPS recommended: DigitalOcean, Linode — \$5/month; or Raspberry Pi at home with port forwarding)
- WireGuard installed: `sudo apt install wireguard`
- Node.js 20+
- Root access (port 53 + WireGuard interface management)

Verification

1. Start server: `sudo npm run dev`
2. Test DNS: `dig @10.13.13.1 instagram.com` from VPN client → returns `0.0.0.0`
3. Test forwarding: `dig @10.13.13.1 google.com` → returns real IPs
4. Open dashboard at `http://<server-ip>:3000`, login with default password `blockme`
5. Add a test device → scan QR code in WireGuard app on phone → confirm connected
6. On phone (connected via VPN): try opening Instagram app → should fail to load
7. Toggle YouTube platform off → confirm YouTube loads; YouTube Shorts URL rule still shows in UI
8. Enable protection mode, try disabling blocking → verify cooldown countdown appears
9. Set a schedule rule → wait for it to activate → confirm blocking auto-engages

Critical Files

- `server/src/dns-server.ts` — core DNS blocking engine
- `server/src/wireguard.ts` — VPN peer management + client config generation
- `server/src/db/schema.ts` — complete data model + seed data
- `server/src/blocklist.ts` — 200+ domain seed list + matching algorithm
- `server/src/routes/settings.ts` — cooldown state machine (hard lock logic)
- `server/src/routes/devices.ts` — device add/remove + QR code generation
- `client/src/pages/DevicesPage.tsx` — QR code setup UI
- `client/src/pages/BlocklistPage.tsx` — granular per-platform UI